

1. JDBC (Database Connectivity)

1. Write a Java program to accept the details of Employee (Eno, EName, Designation, Salary) from a user and store it into the database. (Use Swing)

//Java Code (File Name: EmpForm.java)

```
import javax.swing.*;

import java.awt.event.*;

import java.sql.*;

class EmpForm extends JFrame implements ActionListener{

    JTextField t1,t2,t3,t4; JButton b;

    Connection con; PreparedStatement ps;

    EmpForm(){

        setLayout(null);

        add(new JLabel("Eno")).setBounds(30,30,100,20);

        t1=new JTextField();add(t1).setBounds(150,30,150,20);

        add(new JLabel("Name")).setBounds(30,60,100,20);

        t2=new JTextField();add(t2).setBounds(150,60,150,20);

        add(new JLabel("Designation")).setBounds(30,90,100,20);

        t3=new JTextField();add(t3).setBounds(150,90,150,20);

        add(new JLabel("Salary")).setBounds(30,120,100,20);

        t4=new JTextField();add(t4).setBounds(150,120,150,20);

        b=new JButton("Save");add(b).setBounds(120,160,80,30);

        b.addActionListener(this);

        setSize(350,250);setVisible(true);

        try{

            Class.forName("org.postgresql.Driver");

            con=DriverManager.getConnection("jdbc:postgresql://localhost:5432/postgres","postgres","password");

        }catch(Exception e){System.out.println(e);}

        }

        public void actionPerformed(ActionEvent e){

            try{
```

```

ps=con.prepareStatement("insert into employee values(?,?,?,?)");

ps.setInt(1,Integer.parseInt(t1.getText()));

ps.setString(2,t2.getText());

ps.setString(3,t3.getText());

ps.setDouble(4,Double.parseDouble(t4.getText()));

ps.executeUpdate();

JOptionPane.showMessageDialog(this,"Inserted");

}catch(Exception ex){System.out.println(ex);}

}

}

class Main{

public static void main(String args[]){

new EmpForm();

}

}

```

--Database Code (SQL)

```

CREATE TABLE employee(

eno INT PRIMARY KEY,

ename VARCHAR(50),

designation VARCHAR(50),

salary DOUBLE PRECISION

);

```

2. Write a Java program using JDBC to:

- **Create Product(Pid, Pname, Price) table**
- **Insert at least five records**
- **Display all the records.**

/***** JAVA CODE (FILE NAME: ProductJDBC.java) *****/

```

import java.sql.*;

class ProductJDBC{

public static void main(String args[]){

Connection con=null;

Statement st=null;

try{

Class.forName("org.postgresql.Driver");

con=DriverManager.getConnection("jdbc:postgresql://localhost:5432/postgres","postgres","password");

//create table

st=con.createStatement();

st.executeUpdate("create table if not exists product(pid int primary key,pname varchar(50),price double precision)");

//insert records

st.executeUpdate("insert into product values(1,'Pen',10)");

st.executeUpdate("insert into product values(2,'Book',50)");

st.executeUpdate("insert into product values(3,'Bag',500)");

st.executeUpdate("insert into product values(4,'Pencil',5)");

st.executeUpdate("insert into product values(5,'Scale',20)");

//display records

ResultSet rs=st.executeQuery("select * from product");

while(rs.next()){

System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getDouble(3));

}

con.close();

}catch(Exception e){System.out.println(e);}

}

}

```

/*/ DATABASE CODE (SQL) ***/**

```
CREATE TABLE product(
pid INT PRIMARY KEY,
pname VARCHAR(50),
price DOUBLE PRECISION
);
```

3. Write a Java program using JDBC to display the first record from student() table.

/*/ JAVA CODE (FILE NAME: StudentFirstRecord.java) ***/**

```
import java.sql.*;

class StudentFirstRecord{

public static void main(String args[]){

Connection con=null;

Statement st=null;

try{

Class.forName("org.postgresql.Driver");

con=DriverManager.getConnection("jdbc:postgresql://localhost:5432/postgres","postgres","password");

st=con.createStatement();

ResultSet rs=st.executeQuery("select * from student limit 1");

if(rs.next()){

System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));

}

con.close();

}catch(Exception e){System.out.println(e);}

}

}
```

/*/ DATABASE CODE (SQL) ***/**

```
CREATE TABLE student(

sid INT PRIMARY KEY,
```

```
sname VARCHAR(50),
course VARCHAR(50)
);
```

4. Write a Java program to:

- Create Employee table (ENo, EName, Salary)
- Insert records
- Display all records.

```
/****** JAVA CODE (FILE NAME: EmployeeJDBC.java) *****/
```

```
import java.sql.*;

class EmployeeJDBC{

public static void main(String args[]){

Connection con=null;

Statement st=null;

try{

Class.forName("org.postgresql.Driver");

con=DriverManager.getConnection("jdbc:postgresql://localhost:5432/postgres","postgres","password");

st=con.createStatement();

st.executeUpdate("create table if not exists employee(eno int primary key,ename varchar(50),salary double
precision)");

st.executeUpdate("insert into employee values(1,'Amit',20000)");

st.executeUpdate("insert into employee values(2,'Neha',25000)");

st.executeUpdate("insert into employee values(3,'Ravi',30000)");

st.executeUpdate("insert into employee values(4,'Sita',28000)");

st.executeUpdate("insert into employee values(5,'Rahul',35000)");

ResultSet rs=st.executeQuery("select * from employee");

while(rs.next()){

System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getDouble(3));
```

```

}

con.close();

}catch(Exception e){System.out.println(e);}

}

}

```

/*/ DATABASE CODE (SQL)**

```

CREATE TABLE employee(

eno INT PRIMARY KEY,

ename VARCHAR(50),

salary DOUBLE PRECISION

);

*****/

```

5. Write a Java program to delete employee details using PreparedStatement. Accept employee ID through command line.

/*/ JAVA CODE (FILE NAME: DeleteEmployee.java) *****/**

```

import java.sql.*;

class DeleteEmployee{

public static void main(String args[]){

Connection con=null;

PreparedStatement ps=null;

try{

Class.forName("org.postgresql.Driver");

con=DriverManager.getConnection("jdbc:postgresql://localhost:5432/postgres","postgres","password");

int id=Integer.parseInt(args[0]);

ps=con.prepareStatement("delete from employee where eno=?");

ps.setInt(1,id);

int r=ps.executeUpdate();

if(r>0)System.out.println("Record Deleted");

```

```

else System.out.println("Record Not Found");

con.close();

}catch(Exception e){System.out.println(e);}

}

}

```

/**** DATABASE CODE (SQL)**

```

CREATE TABLE employee(

eno INT PRIMARY KEY,

ename VARCHAR(50),

salary DOUBLE PRECISION

);

*****/

```

6. Write a Java program that establishes JDBC connection with Student database and verifies connection status

/**** JAVA CODE (FILE NAME: CheckConnection.java) *****/**

```

import java.sql.*;

class CheckConnection{

public static void main(String args[]){

Connection con=null;

try{

Class.forName("org.postgresql.Driver");

con=DriverManager.getConnection("jdbc:postgresql://localhost:5432/postgres","postgres","password");

if(con!=null)System.out.println("Connection Successful");

else System.out.println("Connection Failed");

con.close();

}catch(Exception e){System.out.println(e);}

}

}

```

/**** DATABASE CODE (SQL)**

```

CREATE DATABASE student;    */

```

10 Marks**1. Write a Java program to display information about all columns in the DONAR table using ResultSetMetaData.**

/****** JAVA CODE (FILE NAME: DonarMetaData.java) *****/

```
import java.sql.*;

class DonarMetaData{

public static void main(String args[]){

Connection con=null;

Statement st=null;

try{

Class.forName("org.postgresql.Driver");

con=DriverManager.getConnection("jdbc:postgresql://localhost:5432/postgres","postgres","password");

st=con.createStatement();

ResultSet rs=st.executeQuery("select * from donar");

ResultSetMetaData md=rs.getMetaData();

int count=md.getColumnCount();

for(int i=1;i<=count;i++){

System.out.println("Column Name: "+md.getColumnName(i));

System.out.println("Column Type: "+md.getColumnTypeName(i));

System.out.println("Column Size: "+md.getColumnDisplaySize(i));

System.out.println("-----");

}

con.close();

}catch(Exception e){System.out.println(e);}

}

}
```

/****** DATABASE CODE (SQL)

```
CREATE TABLE donar(

id INT PRIMARY KEY,

name VARCHAR(50),
```



```

bloodgroup VARCHAR(5),

city VARCHAR(50)

);

*****/

```

2. Write a Java program to display information about all columns in the DOCTOR table using ResultSetMetaData.

```

/***** JAVA CODE (FILE NAME: DoctorMetaData.java) *****/

import java.sql.*;

class DoctorMetaData{

public static void main(String args[]){

Connection con=null;

Statement st=null;

try{

Class.forName("org.postgresql.Driver");

con=DriverManager.getConnection("jdbc:postgresql://localhost:5432/postgres","postgres","password");

st=con.createStatement();

ResultSet rs=st.executeQuery("select * from doctor");

ResultSetMetaData md=rs.getMetaData();

int count=md.getColumnCount();

for(int i=1;i<=count;i++){

System.out.println("Column Name: "+md.getColumnName(i));

System.out.println("Column Type: "+md.getColumnTypeName(i));

System.out.println("Column Size: "+md.getColumnDisplaySize(i));

System.out.println("-----");

}

con.close();

}catch(Exception e){System.out.println(e);}

}}

```

/*/ DATABASE CODE (SQL)**

```
CREATE TABLE doctor(
id INT PRIMARY KEY,
name VARCHAR(50),
specialization VARCHAR(50),
city VARCHAR(50)
);
*****/
```

2. Servlet 20 Marks

1. Design a Servlet that provides information about HTTP request such as IP address and browser type and server information

```
/***/ JAVA CODE (FILE NAME: RequestInfoServlet.java) *****/
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class RequestInfoServlet extends HttpServlet{

public void doGet(HttpServletRequest req,HttpServletResponse res)throws ServletException,IOException{

res.setContentType("text/html");

PrintWriter out=res.getWriter();

String ip=req.getRemoteAddr();

String browser=req.getHeader("User-Agent");

String serverName=req.getServerName();

int serverPort=req.getServerPort();

out.println("<h2>Client Information</h2>");

out.println("IP Address: "+ip+"<br>");

out.println("Browser: "+browser+"<br>");

out.println("<h2>Server Information</h2>");
```

```

out.println("Server Name: "+serverName+"<br>");
out.println("Server Port: "+serverPort);
}
}

```

2. Write a Java Servlet program named GreetingServlet that reads a name from URL and displays “Hello, [name]! Welcome to Servlet.”

```

/***** JAVA CODE (FILE NAME: GreetingServlet.java) *****/

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class GreetingServlet extends HttpServlet{

public void doGet(HttpServletRequest req,HttpServletResponse res)throws ServletException,IOException{

res.setContentType("text/html");

PrintWriter out=res.getWriter();

String name=req.getParameter("name");

out.println("<h2>Hello, "+name+"! Welcome to Servlet.</h2>");

}

}

```

3. Write a Java Servlet program named ClientInfoServlet that displays client IP address and browser type.

```

/***** JAVA CODE (FILE NAME: ClientInfoServlet.java) *****/

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ClientInfoServlet extends HttpServlet{

public void doGet(HttpServletRequest req,HttpServletResponse res)throws ServletException,IOException{

res.setContentType("text/html");

PrintWriter out=res.getWriter();

```

```
String ip=req.getRemoteAddr();

String browser=req.getHeader("User-Agent");

out.println("<h2>Client Information</h2>");

out.println("IP Address: "+ip+"<br>");

out.println("Browser: "+browser);

}

}
```

4. Write a Servlet named CounterServlet that counts how many times it is accessed and displays the count.

```
import java.io.IOException;

import java.io.PrintWriter;

import javax.servlet.ServletException;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;


// Map the servlet to URL /CounterServlet

@WebServlet("/CounterServlet")

public class CounterServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;


    // Variable to store count of visits

    private int count = 0;


    // Handles GET requests

    protected void doGet(HttpServletRequest request, HttpServletResponse response)

        throws ServletException, IOException {
```

```

// Increment the counter each time the servlet is accessed

count++;

// Set content type to HTML

response.setContentType("text/html");

PrintWriter out = response.getWriter();

// Display the count

out.println("<html><body>");

out.println("<h1>Counter Servlet</h1>");

out.println("<p>This servlet has been accessed <strong>" + count + "</strong> times.</p>");

out.println("</body></html>");

}

// Handle POST requests (optional)

protected void doPost(HttpServletRequest request, HttpServletResponse response)

    throws ServletException, IOException {

    doGet(request, response);

}

}

```

10 Marks

1. Write a Servlet AddServlet that takes two numbers from HTML form and displays the result.

```

<!-- File: add.html -->

<!DOCTYPE html>

<html>

<head>

    <title>Add Two Numbers</title>

</head>

```

```

<body>

  <h2>Add Two Numbers</h2>

  <form action="AddServlet" method="get">

    Number 1: <input type="number" name="num1" required><br><br>

    Number 2: <input type="number" name="num2" required><br><br>

    <input type="submit" value="Add">

  </form>

</body>

</html>

```

// File: AddServlet.java

```

import java.io.IOException;

import java.io.PrintWriter;

import javax.servlet.ServletException;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;


@WebServlet("/AddServlet")

public class AddServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;


    protected void doGet(HttpServletRequest request, HttpServletResponse response)

        throws ServletException, IOException {


        // Read numbers from form

        String num1Str = request.getParameter("num1");

        String num2Str = request.getParameter("num2");

```

```

response.setContentType("text/html");

PrintWriter out = response.getWriter();

out.println("<html><body>");

out.println("<h2>Addition Result</h2>");


try {

    int num1 = Integer.parseInt(num1Str);

    int num2 = Integer.parseInt(num2Str);


    int sum = num1 + num2;

    out.println("<p>" + num1 + " + " + num2 + " = <strong>" + sum + "</strong></p>");

} catch (NumberFormatException e) {

    out.println("<p style='color:red;'>Invalid input! Please enter valid numbers.</p>");

}


// Link back to form

out.println("<br><a href='add.html'>Back to Form</a>");

out.println("</body></html>");

}


protected void doPost(HttpServletRequest request, HttpServletResponse response)

    throws ServletException, IOException {

    doGet(request, response);

}

}

```

2. Write a Servlet `DateTimeServlet` that displays the current date and time when accessed.

```

// File: DateTimeServlet.java

import java.io.IOException;

import java.io.PrintWriter;

```

```
import java.time.LocalDateTime;

import java.time.format.DateTimeFormatter;

import javax.servlet.ServletException;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;


@WebServlet("/DateTimeServlet")

public class DateTimeServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;


    protected void doGet(HttpServletRequest request, HttpServletResponse response)

        throws ServletException, IOException {


        // Get current date and time

        LocalDateTime now = LocalDateTime.now();

        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("EEEE, dd MMMM yyyy HH:mm:ss");

        String formattedDateTime = now.format(formatter);


        // Set response type

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();


        // Display date and time

        out.println("<html><body>");

        out.println("<h2>Current Date and Time</h2>");

        out.println("<p>" + formattedDateTime + "</p>");
```



```

    out.println("</body></html>");
}

protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    doGet(request, response);
}
}

```

3. JSP 20 Marks

1. Write a JSP program which accepts UserName and greets the user according to the server time.

```

<!-- File: greetForm.jsp --%>

<!DOCTYPE html>

<html>

<head>

    <title>User Greeting</title>

</head>

<body>

    <h2>Enter Your Name</h2>

    <form action="greetUser.jsp" method="get">

        User Name: <input type="text" name="userName" required><br><br>

        <input type="submit" value="Greet Me">

    </form>

</body>

</html>

<!-- File: greetUser.jsp --%>

<%@ page import="java.time.LocalTime" %>

```

```
<%@ page import="java.time.format.DateTimeFormatter" %>

<!DOCTYPE html>

<html>

<head>

    <title>Greeting</title>

</head>

<body>

<%

    // Get the user name from the form

    String userName = request.getParameter("userName");

    // Get current server time

    LocalTime currentTime = LocalTime.now();

    // Decide greeting based on server time

    String greeting;

    int hour = currentTime.getHour();

    if(hour >= 5 && hour < 12){

        greeting = "Good Morning";

    } else if(hour >= 12 && hour < 17){

        greeting = "Good Afternoon";

    } else if(hour >= 17 && hour < 21){

        greeting = "Good Evening";

    } else {

        greeting = "Good Night";

    }

    // Format the current time
```

```

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("HH:mm:ss");

String timeString = currentTime.format(formatter);

%>

<h2><%= greeting %>, <%= userName %>!</h2>

<p>Current Server Time: <%= timeString %></p>

<br>

<a href="greetForm.jsp">Back to Form</a>

</body>

</html>

```

2. Write a JSP program to display PATIENT details in tabular form on browser.

```

<%-- File: patientDetails.jsp --%>

<%@ page import="java.util.*" %>

<!DOCTYPE html>

<html>

<head>

    <title>Patient Details</title>

    <style>

        table {

            width: 70%;

            border-collapse: collapse;

        }

        th, td {

            border: 1px solid black;

            padding: 8px;

            text-align: center;

        }

```

```
th {  
    background-color: #f2f2f2;  
}  
  
caption {  
    font-size: 20px;  
    font-weight: bold;  
    margin-bottom: 10px;  
}  
  
</style>  
</head>  
<body>  
<%  
  
    // Sample patient data  
  
    class Patient {  
  
        String id;  
  
        String name;  
  
        int age;  
  
        String gender;  
  
        String disease;  
  
  
        Patient(String id, String name, int age, String gender, String disease) {  
  
            this.id = id;  
  
            this.name = name;  
  
            this.age = age;  
  
            this.gender = gender;  
  
            this.disease = disease;  
  
        }  
  
    }  
}
```

```
// Create sample patients

List<Patient> patients = new ArrayList<>();

patients.add(new Patient("P001", "Rahul Sharma", 30, "Male", "Fever"));

patients.add(new Patient("P002", "Anita Deshmukh", 25, "Female", "Cold"));

patients.add(new Patient("P003", "Sanjay Patil", 40, "Male", "Diabetes"));

patients.add(new Patient("P004", "Pooja Kulkarni", 35, "Female", "Hypertension"));

%>
```

```
<table>

<caption>Patient Details</caption>

<tr>

<th>Patient ID</th>

<th>Name</th>

<th>Age</th>

<th>Gender</th>

<th>Disease</th>

</tr>

<%
```

```
// Display patient details

for(Patient p : patients) {

%>

<tr>

<td><%= p.id %></td>

<td><%= p.name %></td>

<td><%= p.age %></td>

<td><%= p.gender %></td>

<td><%= p.disease %></td>
```

```
</tr>

<%
    }
%>

</table>

</body>

</html>
```

10 Marks

1. Write a JSP program to check voter eligibility

```
<!-- File: voterEligibility.jsp --%>

<!DOCTYPE html>

<html>

<head>

    <title>Voter Eligibility Check</title>

</head>

<body>

    <h2>Voter Eligibility Check</h2>

    <!-- Form to enter age -->

    <form action="voterEligibility.jsp" method="get">

        Enter Your Age:

        <input type="number" name="age" min="0" required>

        <input type="submit" value="Check Eligibility">

    </form>

<%
```

```
// Get age from form

String ageStr = request.getParameter("age");

if(ageStr != null){

    try {

        int age = Integer.parseInt(ageStr);

        // Check voter eligibility

        if(age >= 18){

%>

            <p style="color:green; font-weight:bold;">You are eligible to vote.</p>

<%

        } else {

%>

            <p style="color:red; font-weight:bold;">You are NOT eligible to vote.</p>

<%

        }

    } catch(NumberFormatException e){

%>

        <p style="color:red;">Invalid input! Please enter a valid number.</p>

<%

    }

}

%>

</body>

</html>
```

2. Write a JSP program to check prime number and display result in red color.

```
<!-- File: checkPrime.jsp --%>

<!DOCTYPE html>

<html>

<head>

    <title>Prime Number Check</title>

</head>

<body>

<h2>Check Prime Number</h2>


<form action="checkPrime.jsp" method="get">

    Enter a Number: <input type="number" name="num" required>

    <input type="submit" value="Check">

</form>


<%

    String numStr = request.getParameter("num");

    if(numStr != null){

        try{

            int num = Integer.parseInt(numStr);

            boolean isPrime = true;


            if(num < 2){

                isPrime = false;

            } else {

                for(int i=2;i<=Math.sqrt(num);i++){

                    if(num % i == 0){

                        isPrime = false;

                        break;

                    }

                }

            }

        } catch (NumberFormatException e) {

            // Handle the exception here

        }

    }

    %>
```



```

        }

    }

}

    if(isPrime){
%>

<p style="color:red; font-weight:bold;"><%= num %> is a Prime Number.</p>

<%

        } else {

%>

<p style="color:red; font-weight:bold;"><%= num %> is NOT a Prime Number.</p>

<%

        }

    } catch(NumberFormatException e){

%>

<p style="color:red;">Invalid input!</p>

<%

        }

    }

%>

</body>

</html>

```

3. Write a JSP program to calculate factorial of a number.

```

<%-- File: factorial.jsp --%>

<!DOCTYPE html>

<html>

<head>

    <title>Factorial Calculator</title>

```

```
</head>
```

```
<body>
```

```
<h2>Factorial Calculator</h2>
```

```
<form action="factorial.jsp" method="get">
```

```
    Enter a Number: <input type="number" name="num" min="0" required>
```

```
    <input type="submit" value="Calculate">
```

```
</form>
```

```
<%
```

```
    String numStr = request.getParameter("num");
```

```
    if(numStr != null){
```

```
        try{
```

```
            int num = Integer.parseInt(numStr);
```

```
            long factorial = 1;
```

```
            for(int i=1;i<=num;i++){
```

```
                factorial *= i;
```

```
            }
```

```
%>
```

```
<p>Factorial of <%= num %> is <%= factorial %></p>
```

```
<%
```

```
    } catch(NumberFormatException e){
```

```
%>
```

```
<p style="color:red;">Invalid input!</p>
```

```
<%
```

```
    }
```

```
}
```

```
%>
```

```
</body>
```

```
</html>
```

4. Write a JSP program to check Perfect number (Use Include directive).

```
<%-- File: perfectCheck.jsp --%>
```

```
<%
```

```
    int sum = 0;
```

```
    for(int i=1;i<num;i++){
```

```
        if(num % i == 0){
```

```
            sum += i;
```

```
        }
```

```
    }
```

```
    if(sum == num){
```

```
        out.println("<p>" + num + " is a Perfect Number.</p>");
```

```
    } else {
```

```
        out.println("<p>" + num + " is NOT a Perfect Number.</p>");
```

```
    }
```

```
%>
```

```
<%-- File: perfectNumber.jsp --%>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Perfect Number Check</title>
```

```
</head>
```

```
<body>
```

```
<h2>Perfect Number Check</h2>
```

```
<form action="perfectNumber.jsp" method="get">
```

```

Enter a Number: <input type="number" name="num" min="1" required>

<input type="submit" value="Check">

</form>

```

```

<%

String numStr = request.getParameter("num");

if(numStr != null){

    try{

        int num = Integer.parseInt(numStr);

        request.setAttribute("num", num);

        %><jsp:include page="perfectCheck.jsp"/><%

    } catch(NumberFormatException e){

%>

<p style="color:red;">Invalid input!</p>

<%

    }

}

%>

</body>

</html>

```

5. Write a JSP program to reverse a string.

```

<%-- File: reverseString.jsp --%>

<!DOCTYPE html>

<html>

<head>

    <title>Reverse String</title>

</head>

<body>

```

<h2>Reverse a String</h2>

```
<form action="reverseString.jsp" method="get">
```

```
    Enter a String: <input type="text" name="str" required>
```

```
    <input type="submit" value="Reverse">
```

```
</form>
```

```
<%
```

```
    String str = request.getParameter("str");
```

```
    if(str != null){
```

```
        String reversed = new StringBuilder(str).reverse().toString();
```

```
%>
```

```
<p>Reversed String: <strong><%= reversed %></strong></p>
```

```
<%
```

```
    }
```

```
%>
```

```
</body>
```

```
</html>
```

6. Write a JSP program to calculate Simple Interest.

```
<%-- File: simpleInterest.jsp --%>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Simple Interest Calculator</title>
```

```
</head>
```

```
<body>
```

```
<h2>Simple Interest Calculator</h2>
```

```

<form action="simpleInterest.jsp" method="get">

    Principal: <input type="number" name="p" required><br><br>

    Rate of Interest (%): <input type="number" name="r" required><br><br>

    Time (Years): <input type="number" name="t" required><br><br>

    <input type="submit" value="Calculate">

</form>

```

```

<%

    String pStr = request.getParameter("p");

    String rStr = request.getParameter("r");

    String tStr = request.getParameter("t");

    if(pStr != null && rStr != null && tStr != null){

        try{

            double p = Double.parseDouble(pStr);

            double r = Double.parseDouble(rStr);

            double t = Double.parseDouble(tStr);

            double si = (p * r * t) / 100;

        %>

        <p>Simple Interest = <strong><%= si %></strong></p>

    <%

        } catch(NumberFormatException e){

    %>

    <p style="color:red;">Invalid input!</p>

    <%

        }

    }

```

```
%>
```

```
</body>
```

```
</html>
```

7. Write a JSP program to display digits of a number in words.

```
<%-- File: digitsInWords.jsp --%>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Digits in Words</title>
```

```
</head>
```

```
<body>
```

```
<h2>Digits in Words</h2>
```

```
<form action="digitsInWords.jsp" method="get">
```

```
    Enter a Number: <input type="number" name="num" required>
```

```
    <input type="submit" value="Show Words">
```

```
</form>
```

```
<%
```

```
    String numStr = request.getParameter("num");
```

```
    if(numStr != null){
```

```
        String[] words = {"Zero","One","Two","Three","Four","Five","Six","Seven","Eight","Nine"};
```

```
        char[] digits = numStr.toCharArray();
```

```
%>
```

```
<p>
```

```
<%
```

```
    for(char d : digits){
```

```
        if(Character.isDigit(d)){
```

```

        out.print(words[d - '0'] + " ");
    }

}

%>

</p>

<%

    }

%>

</body>

</html>

```

8. Write a JSP program to find sum of first and last digits.

```

<%-- File: sumFirstLast.jsp --%>

<!DOCTYPE html>

<html>

<head>

    <title>Sum of First and Last Digits</title>

</head>

<body>

<h2>Sum of First and Last Digits</h2>

<form action="sumFirstLast.jsp" method="get">

    Enter a Number: <input type="number" name="num" required>

    <input type="submit" value="Calculate">

</form>

<%

    String numStr = request.getParameter("num");

    if(numStr != null){

```



```

try{

    int num = Integer.parseInt(numStr);

    int last = num % 10;

    int first = num;

    while(first >= 10){

        first /= 10;

    }

    int sum = first + last;

%>

<p>Sum of first and last digits = <strong><%= sum %></strong></p>

<%

    } catch(NumberFormatException e){

%>

<p style="color:red;">Invalid input!</p>

<%

    }

}

%>

</body>

</html>

```

9. Write a JSP program to check voting eligibility using age.

```

<%-- File: voterEligibility.jsp --%>

<!DOCTYPE html>

<html>

<head>

    <title>Voter Eligibility Check</title>

</head>

<body>

```

```
<h2>Voter Eligibility Check</h2>
```

```
<form action="voterEligibility.jsp" method="get">
```

```
    Enter Your Age: <input type="number" name="age" min="0" required>
```

```
    <input type="submit" value="Check">
```

```
</form>
```

```
<%
```

```
    String ageStr = request.getParameter("age");
```

```
    if(ageStr != null){
```

```
        try {
```

```
            int age = Integer.parseInt(ageStr);
```

```
            if(age >= 18){
```

```
%>
```

```
<p style="color:green; font-weight:bold;">You are eligible to vote.</p>
```

```
<%
```

```
        } else {
```

```
%>
```

```
<p style="color:red; font-weight:bold;">You are NOT eligible to vote.</p>
```

```
<%
```

```
        }
```

```
    } catch(NumberFormatException e){
```

```
%>
```

```
<p style="color:red;">Invalid input!</p>
```

```
<%
```

```
    }
```

```
}
```

```
%>
```

</body>

</html>

4. Threads / Multithreading

20 Marks

1. Write a Java program implementing three threads where: ○ first thread generates random number ○ second prints square if even ○ third prints cube if odd.

// File: RandomSquareCube.java

```
import java.util.Random;
```

```
class RandomNumberThread extends Thread {
```

```
    int number;
```

```
    public void run() {
```

```
        Random rand = new Random();
```

```
        number = rand.nextInt(100); // Generate random number 0-99
```

```
        System.out.println("Random Number: " + number);
```

```
    }
```

```
}
```

```
class SquareThread extends Thread {
```

```
    int number;
```

```
    SquareThread(int number) { this.number = number; }
```

```
    public void run() {
```

```
        if(number % 2 == 0)
```

```
            System.out.println("Square of " + number + " = " + (number*number));
```

```
    }
```

```
}
```

```

class CubeThread extends Thread {

    int number;

    CubeThread(int number) { this.number = number; }

    public void run() {

        if(number % 2 != 0)

            System.out.println("Cube of " + number + " = " + (number*number*number));

    }

}

```

```

public class RandomSquareCube {

    public static void main(String[] args) throws InterruptedException {

        RandomNumberThread rThread = new RandomNumberThread();

        rThread.start();

        rThread.join(); // wait for random number

        SquareThread sThread = new SquareThread(rThread.number);

        CubeThread cThread = new CubeThread(rThread.number);

        sThread.start();

        cThread.start();

    }

}

```

2. Write a Java program to define a thread printing text n times (COVID19 and LOCKDOWN2020).

// File: TextPrintThread.java

```

class TextThread extends Thread {

    String text;

    int n;

    TextThread(String text, int n) {

```

```

    this.text = text;

    this.n = n;
}

public void run() {
    for(int i=1;i<=n;i++){
        System.out.println(text);
    }
}
}

```

```

public class TextPrintThread {
    public static void main(String[] args) {
        TextThread t1 = new TextThread("COVID19", 5);
        TextThread t2 = new TextThread("LOCKDOWN2020", 5);

        t1.start();
        t2.start();
    }
}

```

3. Write a program creating 3 threads printing: ○ Hello (10 times) ○ GoodMorning (20 times) ○ HaveANiceDay (30 times)

// File: GreetingThreads.java

```

class HelloThread extends Thread {
    public void run() {
        for(int i=1;i<=10;i++) System.out.println("Hello");
    }
}

```

```
class GoodMorningThread extends Thread {  
  
    public void run() {  
  
        for(int i=1;i<=20;i++) System.out.println("GoodMorning");  
  
    }  
  
}
```

```
class HaveANiceDayThread extends Thread {  
  
    public void run() {  
  
        for(int i=1;i<=30;i++) System.out.println("HaveANiceDay");  
  
    }  
  
}
```

```
public class GreetingThreads {  
  
    public static void main(String[] args) {  
  
        HelloThread t1 = new HelloThread();  
  
        GoodMorningThread t2 = new GoodMorningThread();  
  
        HaveANiceDayThread t3 = new HaveANiceDayThread();  
  
  
        t1.start();  
  
        t2.start();  
  
        t3.start();  
  
    }  
  
}
```

4. Write a program to solve Producer Consumer problem using thread synchronization.

// File: ProducerConsumer.java

```
class Shared {  
  
    int data;  
  
    boolean available = false;
```

```
synchronized void produce(int value) throws InterruptedException {  
    while(!available) wait();  
  
    data = value;  
  
    System.out.println("Produced: " + data);  
  
    available = true;  
  
    notify();  
}  
  
synchronized int consume() throws InterruptedException {  
    while(!available) wait();  
  
    System.out.println("Consumed: " + data);  
  
    available = false;  
  
    notify();  
  
    return data;  
}  
}  
  
class Producer extends Thread {  
    Shared s;  
  
    Producer(Shared s) { this.s = s; }  
  
    public void run() {  
        for(int i=1;i<=10;i++){  
            try {  
                s.produce(i);  
  
                sleep(100); // simulate time  
            } catch(Exception e) {}  
        }  
    }  
}
```

```

    }
}

class Consumer extends Thread {
    Shared s;
    Consumer(Shared s) { this.s = s; }
    public void run() {
        for(int i=1;i<=10;i++){
            try {
                s.consume();
                sleep(150);
            } catch(Exception e) {}
        }
    }
}

```

```

public class ProducerConsumer {
    public static void main(String[] args) {
        Shared s = new Shared();
        new Producer(s).start();
        new Consumer(s).start();
    }
}

```

5. Write a Java program with three threads:

- generate numbers 1–20
- print numbers divisible by 3
- print numbers divisible by 5

// File: DivisibleNumbersThreads.java


```
class GenerateNumbers extends Thread {
```

```
    int[] numbers = new int[20];
```

```
    public void run() {
```

```
        for(int i=1;i<=20;i++){
```

```
            numbers[i-1] = i;
```

```
        }
```

```
    }
```

```
}
```

```
class Div3Thread extends Thread {
```

```
    int[] numbers;
```

```
    Div3Thread(int[] numbers){ this.numbers = numbers; }
```

```
    public void run() {
```

```
        System.out.print("Divisible by 3: ");
```

```
        for(int n : numbers){
```

```
            if(n % 3 == 0) System.out.print(n + " ");
```

```
        }
```

```
        System.out.println();
```

```
    }
```

```
}
```

```
class Div5Thread extends Thread {
```

```
    int[] numbers;
```

```
    Div5Thread(int[] numbers){ this.numbers = numbers; }
```

```
    public void run() {
```

```
        System.out.print("Divisible by 5: ");
```

```
        for(int n : numbers){
```

```
            if(n % 5 == 0) System.out.print(n + " ");
```

```

    }

    System.out.println();

}

}

public class DivisibleNumbersThreads {

    public static void main(String[] args) throws InterruptedException {

        GenerateNumbers g = new GenerateNumbers();

        g.start();

        g.join(); // wait for numbers

        Div3Thread t3 = new Div3Thread(g.numbers);

        Div5Thread t5 = new Div5Thread(g.numbers);

        t3.start();

        t5.start();

    }

}

```

10 Marks

1. Write a Java program to generate 10 random numbers using thread.

// File: RandomNumbersThread.java

```
import java.util.Random;
```

```

class RandomThread extends Thread {

    public void run() {

        Random rand = new Random();

        System.out.println("10 Random Numbers:");

        for(int i=1;i<=10;i++){

```

```

        int num = rand.nextInt(100);

        System.out.print(num + " ");

    }

    System.out.println();

}

}

```

```

public class RandomNumbersThread {

    public static void main(String[] args) {

        RandomThread t = new RandomThread();

        t.start();

    }

}

```

2. Write a Java program to display thread name and priority.

// File: ThreadNamePriority.java

```

class NamePriorityThread extends Thread {

    public void run() {

        System.out.println("Thread Name: " + Thread.currentThread().getName());

        System.out.println("Thread Priority: " + Thread.currentThread().getPriority());

    }

}

```

```

public class ThreadNamePriority {

    public static void main(String[] args) {

        NamePriorityThread t1 = new NamePriorityThread();

        NamePriorityThread t2 = new NamePriorityThread();

        t1.setName("HighPriorityThread");
    }
}

```

```

t2.setName("LowPriorityThread");

t1.setPriority(Thread.MAX_PRIORITY); // 10
t2.setPriority(Thread.MIN_PRIORITY); // 1

t1.start();

t2.start();

}

}

```

3. Write a Java program using thread to display vowels from string every 3 seconds.

// File: VowelThread.java

```

class VowelThread extends Thread {

    String text;

    VowelThread(String text){ this.text = text; }

    public void run() {

        for(char ch : text.toCharArray()){

            if("AEIOUaeiou".indexOf(ch) != -1){

                System.out.println("Vowel: " + ch);

                try { Thread.sleep(3000); } // wait 3 seconds

                catch(InterruptedException e) { e.printStackTrace(); }

            }

        }

    }

}

public class VowelThreadDemo {

    public static void main(String[] args) {

        VowelThread t = new VowelThread("HELLO WORLD JAVA PROGRAMMING");
    }

}

```

```

    t.start();

}

}

```

4. Write a Java program to count even and odd numbers (1–100) using two threads.

// File: EvenOddThreads.java

```

class EvenThread extends Thread {

    public void run() {

        System.out.print("Even Numbers: ");

        for(int i=1;i<=100;i++){

            if(i%2==0) System.out.print(i + " ");

        }

        System.out.println();

    }

}

```

```

class OddThread extends Thread {

    public void run() {

        System.out.print("Odd Numbers: ");

        for(int i=1;i<=100;i++){

            if(i%2!=0) System.out.print(i + " ");

        }

        System.out.println();

    }

}

```

```

public class EvenOddThreads {

    public static void main(String[] args) {

        EvenThread t1 = new EvenThread();
    }

}

```

```

    OddThread t2 = new OddThread();

    t1.start();

    t2.start();

}

}

```

5. Write a Java program to display A–Z alphabets every 2 seconds.

// File: AlphabetThread.java

```

class AlphabetThread extends Thread {

    public void run() {

        for(char ch='A'; ch<='Z'; ch++){

            System.out.print(ch + " ");

            try { Thread.sleep(2000); } // wait 2 seconds

            catch(InterruptedException e) { e.printStackTrace(); }

        }

        System.out.println();

    }

}

public class AlphabetThreadDemo {

    public static void main(String[] args) {

        AlphabetThread t = new AlphabetThread();

        t.start();

    }

}

```

5. Collections Framework

20 Marks

1. Write a Java program to store city names and STD codes using collections and perform add, remove, search.

```
// File: CitySTDCollection.java
```

```
import java.util.*;
```

```
public class CitySTDCollection {
```

```
    public static void main(String[] args) {
```

```
        // Using HashMap for city -> STD code
```

```
        Map<String, Integer> cityMap = new HashMap<>();
```

```
        // Add cities
```

```
        cityMap.put("Pune", 020);
```

```
        cityMap.put("Mumbai", 022);
```

```
        cityMap.put("Nagpur", 0712);
```

```
        cityMap.put("Nashik", 0253);
```

```
        System.out.println("Initial Cities: " + cityMap);
```

```
        // Remove a city
```

```
        cityMap.remove("Nagpur");
```

```
        System.out.println("After Removing Nagpur: " + cityMap);
```

```
        // Search city
```

```
        String searchCity = "Pune";
```

```
        if(cityMap.containsKey(searchCity)){
```

```
            System.out.println(searchCity + " STD Code: " + cityMap.get(searchCity));
```

```
        } else {
```

```
            System.out.println(searchCity + " not found.");
```

```
        }
```

```
}
```

```
}
```

2. Write a Java program to create HashMap and sort it according to keys.

```
// File: HashMapSortByKey.java
```

```
import java.util.*;
```

```
public class HashMapSortByKey {
```

```
    public static void main(String[] args) {
```

```
        HashMap<Integer, String> map = new HashMap<>();
```

```
        map.put(102, "Alice");
```

```
        map.put(101, "Bob");
```

```
        map.put(104, "David");
```

```
        map.put(103, "Carol");
```

```
        System.out.println("Original HashMap: " + map);
```

```
        // Sort by keys using TreeMap
```

```
        TreeMap<Integer, String> sortedMap = new TreeMap<>(map);
```

```
        System.out.println("Sorted by Keys: " + sortedMap);
```

```
    }
```

```
}
```

3. Write a program to load names and phone numbers from text file using hash table.

```
// File: LoadPhoneNumbers.java
```

```
import java.util.*;
```

```
import java.io.*;
```

```
public class LoadPhoneNumbers {
```

```
    public static void main(String[] args) {
```



```

Hashtable<String, String> phoneBook = new Hashtable<>();

try (BufferedReader br = new BufferedReader(new FileReader("phones.txt"))) {
    String line;

    while ((line = br.readLine()) != null) {
        // Expected format: Name,Phone
        String[] parts = line.split(",");
        phoneBook.put(parts[0], parts[1]);
    }
} catch (Exception e) {
    System.out.println("Error: " + e);
}

System.out.println("Phone Book: " + phoneBook);
}
}

```

4. Write a program to load RollNo and Name from text file using hash table.

```

// File: LoadRollNoName.java
import java.util.*;
import java.io.*;

public class LoadRollNoName {
    public static void main(String[] args) {
        Hashtable<Integer, String> students = new Hashtable<>();

        try (BufferedReader br = new BufferedReader(new FileReader("students.txt"))) {
            String line;

            while ((line = br.readLine()) != null) {

```

```

        // Expected format: RollNo,Name

        String[] parts = line.split(",");

        students.put(Integer.parseInt(parts[0]), parts[1]);

    }

} catch(Exception e) {

    System.out.println("Error: " + e);

}

System.out.println("Students Table: " + students);

}

}

```

5. Write a Java program demonstrating ListIterator traversal of ArrayList forward and backward.

```

// File: ArrayListIteratorDemo.java

import java.util.*;

public class ArrayListIteratorDemo {

    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");

        list.add("Banana");

        list.add("Cherry");

        list.add("Date");

        System.out.println("Forward Traversal:");

        ListIterator<String> it = list.listIterator();

        while(it.hasNext()){

            System.out.println(it.next());

        }

    }

}

```

```
System.out.println("Backward Traversal:");  
while(it.hasPrevious()){  
    System.out.println(it.previous());  
}  
}  
}
```

6. Write a Java program to create TreeMap(BookID, BookName) and perform operations.

// File: BookTreeMap.java

```
import java.util.*;
```

```
public class BookTreeMap {
```

```
    public static void main(String[] args) {
```

```
        TreeMap<Integer, String> books = new TreeMap<>();
```

```
        // Add books
```

```
        books.put(103, "Java Programming");
```

```
        books.put(101, "Data Structures");
```

```
        books.put(104, "Operating System");
```

```
        books.put(102, "Database Management");
```

```
        System.out.println("Books TreeMap: " + books);
```

```
        // Remove a book
```

```
        books.remove(104);
```

```
        System.out.println("After removing BookID 104: " + books);
```

```
        // Search book by ID
```

```

int searchId = 102;

if(books.containsKey(searchId)){

    System.out.println("BookID " + searchId + ": " + books.get(searchId));

} else {

    System.out.println("BookID " + searchId + " not found.");

}


// Iterate through TreeMap

System.out.println("Iterating over books:");

for(Map.Entry<Integer, String> entry : books.entrySet()){

    System.out.println("BookID: " + entry.getKey() + ", BookName: " + entry.getValue());

}

}

}

```

7. Write a Java program using LinkedList to add first element, delete last element and display size.

// File: LinkedListExample.java

```
import java.util.LinkedList;
```

```

public class LinkedListExample {

    public static void main(String[] args) {

        // Create a LinkedList of integers

        LinkedList<Integer> list = new LinkedList<>();


        // Add elements

        list.add(10);

        list.add(20);

        list.add(30);
    }
}

```

```
System.out.println("Original List: " + list);

// Add first element
list.addFirst(5);

System.out.println("After adding first element: " + list);

// Delete last element
list.removeLast();

System.out.println("After deleting last element: " + list);

// Display size
System.out.println("Size of the LinkedList: " + list.size());
}
}
```

8. Write a Java program to store words in HashMap and count frequency.

```
// File: WordFrequency.java
import java.util.HashMap;
import java.util.Scanner;

public class WordFrequency {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter a sentence:");
        String sentence = sc.nextLine();

        // Split sentence into words
```

```

String[] words = sentence.split("\\s+");

// Create HashMap to store word frequencies
HashMap<String, Integer> wordCount = new HashMap<>();

// Count frequency
for (String word : words) {
    word = word.toLowerCase(); // optional: ignore case
    if (wordCount.containsKey(word)) {
        wordCount.put(word, wordCount.get(word) + 1);
    } else {
        wordCount.put(word, 1);
    }
}

// Display frequency
System.out.println("Word Frequencies:");
for (String key : wordCount.keySet()) {
    System.out.println(key + " : " + wordCount.get(key));
}

sc.close();
}
}

```

10 Marks

1. Write a Java program to read N names using HashSet and display in ascending order.

// File: HashSetNamesAscending.java

```
import java.util.HashSet;

import java.util.Scanner;

import java.util.TreeSet;


public class HashSetNamesAscending {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter number of names:");

        int N = sc.nextInt();

        sc.nextLine(); // consume newline


        HashSet<String> names = new HashSet<>();


        System.out.println("Enter names:");

        for (int i = 0; i < N; i++) {

            names.add(sc.nextLine());

        }


        TreeSet<String> sortedNames = new TreeSet<>(names);


        System.out.println("Names in ascending order:");

        for (String name : sortedNames) {

            System.out.println(name);

        }


        sc.close();

    }

}
```

2. Write a Java program to create Hashtable for mobile number and student name using Enumeration.

```
// File: HashtableMobileName.java
```

```
import java.util.Hashtable;
```

```
import java.util.Enumeration;
```

```
public class HashtableMobileName {
```

```
    public static void main(String[] args) {
```

```
        Hashtable<Long, String> contacts = new Hashtable<>();
```

```
        contacts.put(9876543210L, "Alice");
```

```
        contacts.put(9123456780L, "Bob");
```

```
        contacts.put(9988776655L, "Charlie");
```

```
        System.out.println("Mobile Number - Student Name:");
```

```
        Enumeration<Long> keys = contacts.keys();
```

```
        while (keys.hasMoreElements()) {
```

```
            Long key = keys.nextElement();
```

```
            System.out.println(key + " - " + contacts.get(key));
```

```
        }
```

```
    }
```

```
}
```

3. Write a Java program to create TreeSet of colors and display ascending order.

```
// File: TreeSetColors.java
```

```
import java.util.TreeSet;
```

```
public class TreeSetColors {
```

```
    public static void main(String[] args) {
```



```
TreeSet<String> colors = new TreeSet<>();
```

```
colors.add("Red");
```

```
colors.add("Blue");
```

```
colors.add("Green");
```

```
colors.add("Yellow");
```

```
colors.add("Orange");
```

```
System.out.println("Colors in ascending order:");
```

```
for (String color : colors) {
```

```
    System.out.println(color);
```

```
}
```

```
}
```

```
}
```

4. Write a Java program to store N integers in LinkedList and display negative integers.

```
// File: LinkedListNegativeIntegers.java
```

```
import java.util.LinkedList;
```

```
import java.util.Scanner;
```

```
public class LinkedListNegativeIntegers {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Enter number of integers:");
```

```
        int N = sc.nextInt();
```

```
        LinkedList<Integer> numbers = new LinkedList<>();
```

```
        System.out.println("Enter integers:");
```

```

    for (int i = 0; i < N; i++) {
        numbers.add(sc.nextInt());
    }

    System.out.println("Negative integers in the list:");
    for (int num : numbers) {
        if (num < 0) {
            System.out.println(num);
        }
    }

    sc.close();
}
}

```

5. Write a Java program to store N integers in collection without duplicates and display sorted order.

// File: CollectionNoDuplicatesSorted.java

```

import java.util.Scanner;
import java.util.Set;
import java.util.TreeSet;

public class CollectionNoDuplicatesSorted {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter number of integers:");
        int N = sc.nextInt();

        Set<Integer> numbers = new TreeSet<>(); // TreeSet: no duplicates + sorted
    }
}

```

```
System.out.println("Enter integers:");
```

```
for (int i = 0; i < N; i++) {
    numbers.add(sc.nextInt());
}
```

```
System.out.println("Integers without duplicates in sorted order:");
```

```
for (int num : numbers) {
    System.out.println(num);
}
```

```
sc.close();
```

```
}
```

```
}
```

6. Create hash table maintaining mobile number and student name contact list.

```
// File: HashtableContactList.java
```

```
import java.util.Hashtable;
```

```
import java.util.Scanner;
```

```
import java.util.Enumuration;
```

```
public class HashtableContactList {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        Hashtable<Long, String> contacts = new Hashtable<>();
```

```
        System.out.println("Enter number of contacts:");
```

```
        int N = sc.nextInt();
```

```
        sc.nextLine(); // consume newline
```

```
for (int i = 0; i < N; i++) {  
    System.out.println("Enter mobile number:");  
    long mobile = sc.nextLong();  
    sc.nextLine();  
    System.out.println("Enter student name:");  
    String name = sc.nextLine();  
  
    contacts.put(mobile, name);  
}  
  
System.out.println("Contacts:");  
Enumeration<Long> keys = contacts.keys();  
while (keys.hasMoreElements()) {  
    Long key = keys.nextElement();  
    System.out.println(key + " - " + contacts.get(key));  
}  
  
sc.close();  
}  
}
```

6. Spring

10 Marks

1. Write a program to display the current date using Spring.

// File: CurrentDateSpring.java

```
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
import java.util.Date;
```

```
public class CurrentDateSpring {  
    public static void main(String[] args) {  
        // Load Spring configuration file  
  
        ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");  
  
        // Get the date bean  
  
        Date currentDate = (Date) context.getBean("currentDate");  
  
        System.out.println("Current Date: " + currentDate);  
    }  
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
    http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<!-- Bean to provide current date -->
```

```
<bean id="currentDate" class="java.util.Date"/>
```

```
</beans>
```